

Project Methods

Table of contents

Introduction	1
Methods	1
Data and Tools	1
Procedure (step-by-step with rationale)	2
Stage 1: Extract topology (<code>analyze_snapshot.py</code>)	2
Stage 2: Slab clip and visualization (<code>batch_clip.py</code>)	3
Stage 3: Topology overlap stats (<code>ndtopo_stats.py</code>)	4
Stage 4: Batch crops and slabs (<code>batch_crop_and_clip.py</code>)	4
Data Analysis Outputs (what we inspected)	5
Troubleshooting (common issues seen)	5
Materials / Sources	5

Introduction

This project extracts, visualizes, and quantifies cosmic-web topology (walls and filaments) from large N-body snapshots using DisPerSE-based workflows. We combine command-line scripts to: - Stream/crop particle data, tessellate density, and extract Morse–Smale structures. - Clip slabs and flatten outputs for 2D overlays and PNGs. - Compute overlap/unique/unassigned topology statistics using authoritative Delaunay scalars. - Batch these steps over many crop tiles and slab positions for large volumes.

Methods

Data and Tools

- **Data:** Quijote snapshot (e.g., `data/snap_010.hdf5`), 1 Gpc/h box, `PartType1` dark matter coordinates.
- **Core tools:**
 - DisPerSE binaries: `delaunay_3D`, `mse`, `netconv`, `skelconv` (installed in your environment or pointed via `--disperse-bin-dir`).
 - ParaView `pvpython` (for `batch_clip.py` rendering).
 - Python scripts in `scripts/`: `analyze_snapshot.py`, `batch_clip.py`, `batch_crop_and_clip.py`, `ndtopo_stats.py`.

- **Dependencies:** `vtkmodules` (for VTK I/O), `hdf5plugin` (Blosc filter), `numpy`, `h5py`.
- **Units:** Snapshot in `kpc/h`; outputs typically in `mpc/h` after scaling. Cropped runs rebase to the crop origin for periodic tessellation, then shift VTK outputs back to global coordinates.

Procedure (step-by-step with rationale)

Stage 1: Extract topology (`analyze_snapshot.py`)

- **Purpose:** Build density (DTFE) and extract walls/filaments from a cropped region.
- **Key inputs:** Snapshot HDF5; crop box (`--crop-box`); stride or target count; DisPerSE binaries; output directory/prefix.
- **How it works:**
 - Coordinate streaming & decimation
 - * Read coordinates in chunks (default 2M) to avoid loading all 134M particles.
 - * Apply crop (e.g., 500000–1000000 `kpc/h` `x/y`, 100000–200000 `kpc/h` `z`) to keep the region of interest.
 - * With `--stride 1`, keep ~3.47M points (down from 134M) to avoid memory blow-ups.
 - * Optionally set `--target-count 2000000` to auto-compute a stride that keeps ~2M points by taking every Nth particle.
 - Rebase crop for periodic tessellation
 - * Subtract crop minima so coords start at (0,0,0); set NDfield BBOX to crop size.
 - * Keeps `delaunay_3D -periodic` wrapping only within the crop; store crop origin for shifting VTKs back later.
 - Delaunay tessellation (DTFE density)
 - * Run `delaunay_3D` on the rebased NDfield with `--periodic / --delaunay-btype periodic`.
 - * Produces a Delaunay tessellation and DTFE density per vertex/cell; this density is what MSE uses.
 - * Persistence is not applied here; this is a pure geometric tessellation.
 - MSE (topology extraction)
 - * Run `mse` on the Delaunay NDnet with significance (`--mse-nsig`), manifold tag (`--dump-manifolds`, e.g., `JE1a`), and arc tag (`--dump-arcs`, e.g., `U`).
 - * Persistence: `--mse-nsig 3.0` or `5.0` (higher sigma prunes low-significance features; ratios printed in logs).
 - * Dumps:
 - `--dump-manifolds JE1a` → descending manifolds (walls).
 - `--dump-arcs U` → ascending arcs (filaments).
 - * Optional `--mse-vertex-as-minima` (off here) would force ver-

- tices to be minima.
 - Conversion & shift back
 - * Convert manifolds/skeletons with `netconv/skelconv`; optional smoothing (`--netconv-smooth 20, --skelconv-smooth 20`).
 - * Shift exported VTKs (Delaunay, manifolds, filaments) back to global coordinates using the stored crop origin; NDnet/NDskl stay rebased.
 - Summaries/outputs
 - * Write `<prefix>_summary_stats.csv` (scalar stats for each VTK).
 - * Outputs: rebased NDnet/NDskl; shifted VTKs (`_delaunay[.S###].vtu, _manifolds_<TAG>[.S###].vtu, _filaments_<ARC>[.S###].vtp`).
- **Why:** Reduces the dataset to a safe size, builds the DTFE density that MSE needs, applies persistence filtering for meaningful walls/filaments, and delivers globally aligned VTKs for the next stages.

Stage 2: Slab clip and visualization (`batch_clip.py`)

- **Purpose:** Slice each crop into thin slabs, flatten to 2D, and visualize overlays.
- **Key inputs:** Shifted VTKs from Stage 1 (walls, filaments, density); slab axis/origin/thickness; resample dimensions; scalar name.
- **How it works:**
 - Clip slab
 - * Load shifted walls/filaments/density; clip to a slab (e.g., `--slab-axis z, --slab-origin 0/10/50, --slab-thickness 10 mpc/h`).
 - * Uses clipped bounds so resampling never samples outside the crop.
 - Flatten / average / resample
 - * Flatten walls/filaments onto the slab plane (set slab axis to zero).
 - * Average density across the slab thickness; resample to a regular grid (`--resample-dims 512×512×64` or `500×500×100`).
 - * Choose scalar to render/average (`--scalar-name log_field_value` commonly).
 - Render PNGs (optional)
 - * 3D slices and 2D flats for walls, filaments, density; composites layer density with walls/filaments.
 - * PNG settings: `--save-pngs; --png-resolution` (default `1600×1200`); `--composite-opacity` (default `0.6`).
 - Outputs per slab
 - * VTK: `_walls_3d.vtu, _filaments_3d.vtp, _density_3d.vtu/.vti; flattened _walls.vtu, _filaments.vtp, _density.vtu, _density_avg.vti, _walls_filaments.vtm`.
 - * Stats: `<prefix>_summary_stats.csv`; PNGs if requested.
- **Why:** Delivers slab-aligned 2D/3D products so you can layer density,

walls, and filaments in 2D to see where structures overlap or stay separate.

Stage 3: Topology overlap stats (`ndtopo_stats.py`)

- **Purpose:** Quantify overlap vs uniqueness of walls/filaments using authoritative Delaunay scalars.
- **Key inputs:** Shifted Delaunay VTU (`true_index`, scalars `mass/field/log_field`), walls VTU (`true_index`), filaments VTP (cell integer part). Native NDnet/NDskl allowed with `--write-vtk`.
- **How it works:**
 - Optionally convert native files with `--write-vtk` (unsmoothed) via `netconv/skelconv`.
 - Match IDs:
 - * Walls Delaunay: `true_index`.
 - * Filaments Delaunay: `int(cell)` (critical point index); interpolated arc samples have IDs not present in Delaunay.
 - Aggregate Delaunay scalars over categories: `shared` (wall filament), `wall_only`, `filament_only`, `unassigned`.
 - Settings: `--walls-id-field true_index`, `--filaments-id-field cell`, `--scalar-fields mass field_value log_field_value`; add `--netconv-bin/--skelconv-bin` when paths are needed.
- **Output:** `<prefix>_topology_stats.csv` with counts and `<scalar>_sum/<scalar>_mean` per category.
- **Why:** Shows how much mass/field value sits in overlapping vs unique topology membership. Filament counts can exceed matched mass because many filament samples are interpolated along arcs and have no Delaunay node.

Stage 4: Batch crops and slabs (`batch_crop_and_clip.py`)

- **Purpose:** Automate stages 1–3 over many crop tiles and slab positions.
- **Key inputs:** Snapshot; crop size/ranges; slab step/thickness; persistence/smoothing settings; resample dims; scalar name.
- **How it works:**
 - Tile snapshot
 - * Set `--crop-size` (e.g., `500000×500000×100000` kpc/h) and ranges (`--x-range`, `--y-range`, `--z-range`).
 - * Iterate non-overlapping tiles; skip empty/partial edge tiles.
 - Per-crop actions
 - * Run Stage 1 with chosen persistence and smoothing (e.g., `--mse-nsig 5.0`, `--dump-manifolds JE1a`, `--dump-arcs U`, `--netconv-smooth 20`, `--skelconv-smooth 20`).
 - * Run Stage 3 to write `<crop_prefix>_topology_stats.csv`.
 - * Derive slab origins from `--slab-step/--slab-thickness`; run Stage 2 for each slab (`--resample-dims` often `500×500×100` or `512×512×64`; `--scalar-name log_field_value`).

- Output layout
 - * `output-root/crop_x.../`: NDnet/NDskl (rebased), shifted VTKs, per-crop topology stats.
 - * `slab_z.../` subfolders: per-slab VTKs, stats, and PNGs.
- **Why:** Runs the full pipeline over many tiles and slab positions, keeping outputs per-crop/per-slab for clean aggregation and visualization.

Data Analysis Outputs (what we inspected)

- **VTK files:** shifted walls/filaments/density (3D and flattened 2D), plus averaged density grids for slice/overlay inspection.
- **PNGs:** 3D slices, 2D flats, and composites that show layered density/walls/filaments to visually assess overlap and separation.
- **Summary stats:** Per-file scalar stats (`*_summary_stats.csv`) and per-crop topology overlap stats (`*_topology_stats.csv`) to quantify counts, sums, and means in shared/unique/unassigned categories.

Troubleshooting (common issues seen)

- **VTK import errors:** Ensure `vtkmodules` is on `PYTHONPATH` (e.g., via conda environment hooks) and ParaView’s Python is discoverable when using `pvpython`.
- **HDF5 Blosc filter:** Install `hdf5plugin` in the environment that runs `analyze_snapshot.py`; otherwise HDF5 load fails.
- **Interpolated filament IDs:** `filament_only` counts can exceed mass because many filament samples are interpolated along arcs and have no Delaunay node; this is expected.
- **Clipping artifacts:** Use slab bounds for resampling (`batch_clip.py` already does) and verify crop rebasing/shift if density extends beyond the tile.
- **File naming:** Stems no longer duplicate format tokens; if older files remain, prefer the non-duplicated names for downstream steps.

Materials / Sources

- Quijote snapshot (local: `data/snap_010.hdf5`).
- DisPerSE binaries: <https://www.iap.fr/user/iap/sousbie/web/html/indexc1d6.html> (or your install location).
- ParaView `pvpython`: <https://www.paraview.org/>.
- Python scripts: in `scripts/` (local repository).
- Dependencies: `hdf5plugin` (Blosc filter), `vtkmodules`, `numpy`, `h5py` (from your Python environment/conda).